# Overview of Proximal policy optimization

*Subject: Proximal policy optimization*

---

**1.**

Policy gradient laws: the advantage function The advantage function (denoted as A {\displaystyle A} ) is central to PPO, as it tries to answer the question of whether a specific action of the agent is better or worse than some other possible action in a given state. By definition, the advantage function is an estimate of the relative value for a selected action. If the output of this function is positive, it means that the action in question is better than the average return, so the possibilities of selecting that specific action will increase. The opposite is true for a negative advantage output. The advantage function can be defined as A = Q − V {\displaystyle A=Q-V} , where Q {\displaystyle Q} is the discounted sum of rewards (the total weighted reward for the completion of an episode) and V {\displaystyle V} is the baseline estimate. Since the advantage function is calculated after the completion of an episode, the program records the outcome of the episode. Therefore, calculating advantage is essentially an unsupervised learning problem. The baseline estimate comes from the value function that outputs the expected discounted sum of an episode starting from the current state. In the PPO algorithm, the baseline estimate will be noisy (with some variance), as it also uses a neural network, like the policy function itself. With Q {\displaystyle Q} and V {\displaystyle V} computed, the advantage function is calculated by subtracting the baseline estimate from the actual discounted return. If A > 0 {\displaystyle A>0} , the actual return of the action is better than the expected return from experience; if A < 0 {\displaystyle A<0} , the actual return is worse.

**2.**

for k = 0 , 1 , 2 , … {\textstyle k=0,1,2,\ldots } do Collect set of trajectories D k = { τ i } {\textstyle {\mathcal {D}}_{k}=\left\{\tau _{i}\right\}} by running policy π k = π ( θ k ) {\textstyle \pi _{k}=\pi \left(\theta _{k}\right)} in the environment. Compute rewards-to-go R ^ t {\textstyle {\hat {R}}_{t}} . Compute advantage estimates, A ^ t {\textstyle {\hat {A}}_{t}} (using any method of advantage estimation) based on the current value function V φ k {\textstyle V_{\phi _{k}}} . Estimate policy gradient as g ^ k = 1 | D k | ∑ τ ∈ D k ∑ t = 0 T ∇ θ log ■ π θ ( a t ■ s t ) | θ k A ^ t {\displaystyle {\hat {g}}_{k}=\left.{\frac {1}{\left|{\mathcal {D}}_{k}\right|}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\nabla _{\theta }\log \pi _{\theta }\left(a_{t}\mid s_{t}\right)\right|_{\theta _{k}}{\hat {A}}_{t}}

**3.**

Simplicity PPO approximates what TRPO does, with considerably less computation. It uses first-order optimization (the clip function) to constrain the policy update, while TRPO uses KL divergence constraints (second-order optimization). Compared to TRPO, the PPO method is relatively easy to implement and requires less computational resource and time. Therefore, it is cheaper and more efficient to use PPO in large-scale problems.

**4.**

Use the conjugate gradient algorithm to compute x ^ k ≈ H ^ k − 1 g ^ k {\displaystyle {\hat {x}}_{k}\approx {\hat {H}}_{k}^{-1}{\hat {g}}_{k}} where H ^ k {\textstyle {\hat {H}}_{k}} is the

Hessian of the sample average KL-divergence. Update the policy by backtracking line search with θ k + 1 = θ k + α j 2 δ x ^ k T H ^ k x ^ k x ^ k {\displaystyle \theta _{k+1}=\theta _{k}+\alpha ^{j}{\sqrt {\frac {2\delta }{{\hat {x}}_{k}^{T}{\hat {H}}_{k}{\hat {x}}_{k}}}}{\hat {x}}_{k}} where j ∈ { 0 , 1 , 2 , … K } {\textstyle j\in \{0,1,2,\ldots K\}} is the smallest value which improves the sample loss and satisfies the sample KL-divergence constraint. Fit value function by regression on mean-squared error: φ k + 1 = arg ■ min φ 1 | D k | T ∑ τ ∈ D k ∑ t = 0 T ( V φ ( s t ) − R ^ t ) 2 {\displaystyle \phi _{k+1}=\arg \min _{\phi }{\frac {1}{\left|{\mathcal {D}}_{k}\right|T}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\left(V_{\phi }\left(s_{t}\right)-{\hat {R}}_{t}\right)^{2}} typically via some gradient descent algorithm.

## 5.

Stability While other RL algorithms require hyperparameter tuning, PPO comparatively does not require as much (0.2 for epsilon can be used in most cases). Also, PPO does not require sophisticated optimization techniques. It can be easily practiced with standard deep learning frameworks and generalized to a broad range of tasks.

## 6.

for k = 0 , 1 , 2 , … {\textstyle k=0,1,2,\ldots } do Collect set of trajectories D k = { τ i } {\textstyle {\mathcal {D}}_{k}=\left\{\tau _{i}\right\}} by running policy π k = π ( θ k ) {\textstyle \pi _{k}=\pi \left(\theta _{k}\right)} in the environment. Compute rewards-to-go R ^ t {\textstyle {\hat {R}}_{t}} . Compute advantage estimates, A ^ t {\textstyle {\hat {A}}_{t}} (using any method of advantage estimation) based on the current value function V φ k {\textstyle V_{\phi _{k}}} . Update the policy by maximizing the PPO-Clip objective: θ k + 1 = arg ■ max θ 1 | D k | T ∑ τ ∈ D k ∑ t = 0 T min ( π θ ( a t ■ s t ) π θ k ( a t ■ s t ) A π θ k ( s t , a t ) , g ( ■ , A π θ k ( s t , a t ) ) ) {\displaystyle \theta _{k+1}=\arg \max _{\theta }{\frac {1}{\left|{\mathcal {D}}_{k}\right|T}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\min \left({\frac {\pi _{\theta }\left(a_{t}\mid s_{t}\right)}{\pi _{\theta _{k}}\left(a_{t}\mid s_{t}\right)}}A^{\pi _{\theta _{k}}}\left(s_{t},a_{t}\right),\quad g\left(\epsilon ,A^{\pi _{\theta _{k}}}\left(s_{t},a_{t}\right)\right)\right)} typically via stochastic gradient ascent with Adam. Fit value function by regression on mean-squared error: φ k + 1 = arg ■ min φ 1 | D k | T ∑ τ ∈ D k ∑ t = 0 T ( V φ ( s t ) − R ^ t ) 2 {\displaystyle \phi _{k+1}=\arg \min _{\phi }{\frac {1}{\left|{\mathcal {D}}_{k}\right|T}}\sum _{\tau \in {\mathcal {D}}_{k}}\sum _{t=0}^{T}\left(V_{\phi }\left(s_{t}\right)-{\hat {R}}_{t}\right)^{2}} typically via some gradient descent algorithm. Like all policy gradient methods, PPO is used for training an RL agent whose actions are determined by a differentiable policy function by gradient ascent. Intuitively, a policy gradient method takes small policy update steps, so the agent can reach higher and higher rewards in expectation. Policy gradient methods may be unstable: A step size that is too big may direct the policy in a suboptimal direction, thus having little possibility of recovery; a step size that is too small lowers the overall efficiency. To solve the instability, PPO implements a clip function that constrains the policy update of an agent from being too large, so that larger step sizes may be used without negatively affecting the gradient ascent process.